

# Shell-Instrument Footguns – the § . . ( ) control-needle checklist

| Field         | Value                                                                                                                                                         |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Revision      | 3                                                                                                                                                             |
| Created       | 2026-07-22                                                                                                                                                    |
| Last modified | 2026-07-22T21:02:04Z                                                                                                                                          |
| Status        | active                                                                                                                                                        |
| Authority     | Constitution §11.4.201(7)(12) + §11.4.67(6). Consulted by EVERY §11.4.142/§11.4.209 code review that touches a shell-based test, gate, guard, or measurement. |
| Scope         | Universal (§11.4.17) — inherited by reference (§11.4.28/§11.4.177), never copied per project.                                                                 |

## Purpose

Every entry below is a shell construct that returns a **clean, confident, WRONG answer without crashing** — so §11.4.1's script-bug clause (crashes-into-FAIL) never fires and §11.4.67's parse-check (parse-time only) cannot see it. Each entry was either captured in a real forensic incident or re-demonstrated under control conditions (dates cited). Per §11.4.201(7)(b), **no zero / absence / PASS produced by a shell instrument is evidence until a class-matched control needle has proven the instrument can see through the SAME path.**

### Rule of use (binding):

1. Before trusting ANY zero/absence/PASS from a shell measurement, run a known-positive control needle through the SAME instrument, path, and query class (§11.4.201(7)(b)).
2. Every code review of shell-based tests/gates/guards/measurements consults this checklist and records the consultation (§11.4.201(12), gate invariant (vii)).
3. This is a LIVING document: a newly-measured footgun class is APPENDED with its captured demonstration in the same change-window (§11.4.5) — never carried as tribal memory (§11.4.215: the binding reference lives tracked, in-repo).

## Part A – Instrument-side footguns (the measurement lies)

### I . Exec-wrapper commands cannot run shell functions or aliases

- **Construct:** `timeout <sec> my_shell_function , nice -n N my_alias , env VAR=x my_function , xargs , watch , sudo` — any wrapper that `exec` s its argument as a BINARY.
- **Wrong confident answer:** `rc=127 failed to run command '...': No such file or directory` — read as "the function failed / hung / is broken" while the function is healthy. A §11.4.201(1) FALSE-POSITIVE refusal when used inside a guard; a false FAIL when used inside a test.
- **Demonstrated:** 2026-07-23 control experiment — `timeout 1 myfunc` on a defined, working function → `rc=127`.
- **Countermeasure:** wrap in a shell: `timeout 1 bash -c 'source lib.sh; myfunc'` ; or export the function ( `export -f myfunc; timeout 1 bash -c myfunc` ); or bound the wait with a polling loop in the current shell.
- **Control needle:** run the wrapper against a trivially-working function first ( `timeout 5 true_function` ); `rc=127` on the needle proves the INSTRUMENT cannot see functions — the measurement says nothing about the function under test.

## I . `pgrep -f` / `pkill -f` match the probing script's own command line (carrier match)

- **Construct:** `pgrep -f '<pattern>'` as a "is X running?" probe.
- **Wrong confident answer:** a non-empty match (`rc=0`) whose only hits are CARRIERS — the probing script itself, a harness/heredoc quoting the pattern, an agent whose prompt embeds the token, a monitor mentioning it (§11.4.196(D) / §12.12 / §11.4.201(7)(a)). Inverted risk: `pkill -f` kills the prober or an innocent carrier.
- **Demonstrated:** 2026-07-23 control experiment — `pgrep -fa` `UNIQUE_NEEDLE_XYZZY_42` with ZERO real processes matched exactly ONE pid: the probe's own harness command line. Forensic incidents: the 2026-07-13 build-guard false-refusing every spawn (§11.4.201 forensic anchor (a)); the §12.12 self-kill caution.
- **Countermeasure:** re-read each matched PID's real `/proc/<pid>/cmdline` ; exclude `$$ / $PPID` /known carrier classes; assert the matched process IS the thing (`argv[0]/exe` identity), not a mention of it.
- **Control needle:** positive — a real instance of the target must match; negative — with the target provably absent, the probe must return ZERO after carrier exclusion (a hit then = carrier leak in the exclusion).

## I . Structural-delimiter text extractors truncate the unit under test (FALSE PASS)

- **Construct:** extracting a function/block for isolated testing with a delimiter-keyed scanner, canonically `awk` terminating at the first column-0 `}` ( `/^\}/` ).
- **Wrong confident answer:** when the body legitimately contains the delimiter at column 0 (heredoc content, quoted string, embedded doc), the extractor stops EARLY and hands the test a PARTIAL artifact; the test then passes/fails against code that is not the code under test — a FALSE PASS when the truncated tail carried the behaviour being verified (observed in-session 2026-07-22 as a `stderr-test` false PASS; class re-demonstrated 2026-07-23: a heredoc containing a column-0 `}` made the extractor drop the function's tail marker entirely).

- **Countermeasure:** extract with a real parser path ( `declare -f name` after sourcing in a sandbox shell; or `bash -c 'source f; type name'` ), never a text scanner; if text extraction is unavoidable, assert the extraction's LAST line is the expected terminator AND that a tail-sentinel present in the source survived extraction.
- **Control needle:** plant a unique sentinel as the LAST body line of the unit in a fixture copy; the extraction MUST contain it. Sentinel absent  $\Rightarrow$  the extractor is truncating and every result derived from it is void.

## I . `bash -x` (xtrace) is blinded when the code under test redirects stderr

- **Construct:** tracing code under test with `set -x` / `bash -x` while that code executes `exec 2>/dev/null` (or any stderr re-target) — xtrace writes to stderr by default, so the code under test SILENCES its own trace mid-stream.
- **Wrong confident answer:** the trace simply STOPS; the missing tail is read as "nothing further executed" / "it hung here" — a §11.4.201(6) FALSE-NULL (the instrument went blind; the code kept running).
- **Demonstrated:** 2026-07-22 forensic incident (the sourced- `exec` defect of §11.4.67(6) blinded its own investigation); 2026-07-23 control experiment — a subshell running `set -x; exec 2>/dev/null; <more commands>` yielded exactly ONE captured trace line ( `+ exec` ) and then silence while the commands ran.
- **Countermeasure:** route the trace to a dedicated fd the code under test does not own: `exec 7>>/path/trace; BASH_XTRACEFD=7; set -x` — the real 2026-07-22 investigation's working instrument (per-step `EPOCHREALTIME` timing survived the code's stderr games).
- **Control needle:** the trace MUST contain a marker command executed AFTER the code under test's last redirection; marker absent  $\Rightarrow$  the trace's silence is instrument blindness, not program state.

## I . Pipeline exit status is the LAST stage's (verdict discarded)

- **Construct:** `checker | head` , `checker | tee` , `checker | grep -c ...` used where `checker` 's exit code is the verdict.

- **Wrong confident answer:** the pipeline returns the PAGER/FILTER's status — "clean ✓" from a checker that failed or never ran (forensic FACT, 2026-07-17 harvest: a parse-check said "clean" three times without checking anything).
- **Countermeasure:** `set -o pipefail` (bash), or capture the checker's status explicitly ( `checker > out; rc=$?` ), or use `PIPESTATUS` .
- <sup>6</sup> • **Control needle:** run a KNOWN-FAILING input through the same pipeline; if the pipeline still exits 0, the verdict path is broken.

## I . Regex/quoting dialect drift (the query searches a string nobody wrote)

- **Construct:** `\|` under ERE (literal pipe, not alternation); escapes consumed by an intervening/wrapper shell; line-start anchors missing indented/colour-escaped lines; multi-byte-splitting binary extractors (forensic FACTs, 2026-07-17 harvest, all real).
- **Wrong confident answer:** a quiet zero — read as absence — from a query that could never match anything.
- **Countermeasure + needle:** §11.4.201(7)(b)(c) verbatim — the control needle MUST carry the SAME load-bearing query features (dialect constructs, anchoring, quoting, encoding); a bare-literal needle certifies nothing about an alternation/anchored/encoded query.

## I . A watchdog subshell spawned inside `$(...)` holds the pipe write-end – the substitution stalls to the FULL budget after instant work

- **Construct:** a background watchdog / timeout subshell started INSIDE code whose output is captured via command-substitution — `out=$(probe)` where `probe` runs ( `sleep "$budget"; kill ...` ) & `wd=$!` as its bound and later disarms it early with `kill "$wd"` .
- **Wrong confident answer:** the `$(...)` read returns only at EOF on its pipe, and EOF requires EVERY write-end fd closed — the watchdog subshell AND its external- `sleep` child inherited that write-end at spawn; killing the subshell ORPHANS the `sleep` , which keeps the fd open until the budget elapses (demonstrated in bash, where `sleep` forks as an external child; in shells whose `sleep` is a builtin the subshell itself sleeps and dies with the disarm —

the GENERAL class, any grandchild holding the write-end, remains). The substitution stalls for the FULL timeout budget even though the real work completed instantly, with every verdict and exit code CORRECT — read as "the probe is slow / hit its timeout / the host is wedged" while everything is healthy (a §11.4.201(1)-class false signal at the TIMING layer; no verdict-checking gate can see it because no verdict is wrong).

- **Demonstrated:** forensic incident measured 2026-07-23 (a consuming toolchain's install re-review): a fast-stub probe read via command-substitution took 15s — exactly the watchdog budget — while the SAME probe writing to a file took 0s; every install/verify seam stalled +15s on a perfectly healthy host. Control re-demonstration run 2026-07-22T20:36Z UTC (transcript-captured 20:41Z), 3/3 deterministic iterations: naive `$(...)` capture 5006–5007 ms == the full 5000 ms budget on instant work; the SAME probe to a file 43–46 ms (the pipe IS the mechanism); the fd-redirected watchdog 43–45 ms with output intact; a genuinely-wedged probe under the fixed watchdog still bounded at `rc=124` ≈ budget (the protective function survives the fix). Repro caution, itself measured: an immediate disarm can RACE the watchdog before it forks its `sleep` — no orphan, no stall; a deterministic repro waits for the grandchild (`pgrep -P "$wd"` — parent-PID match, never cmdline, per I2) before disarming, mirroring real work taking >0 time.
- **Countermeasure:** redirect the watchdog subshell's fds away from the cmd-subst pipe at spawn — `( sleep "$budget"; kill ... ) >/dev/null 2>&1 &` — or have the probe write its output to a FILE the caller reads instead of being captured via `$(...)`. Both preserve the watchdog's bound PROVIDED the watchdog kills the WORK process (or its process group), never merely the enclosing probe shell: a watchdog that kills only the shell leaves a wedged external child holding the write-end, and the stall becomes the CHILD's lifetime — unbounded by the budget. With the work as the kill target, genuinely-wedged work is still killed at budget with `rc=124`.
- **Control needle:** time a KNOWN-instant probe through the SAME `$(...)` capture path; elapsed ≈ the watchdog budget ⇒ the stall is the instrument's own pipe topology, NOT the probed work — the timing measurement says nothing about the work until the needle reads ~0.

## Part B – Code-side footguns (the shipped shell code lies to its host)

### C . Bare `exec` with side redirections in a SOURCED function mutates the caller's shell permanently

- **Construct:** `exec 9>>"$file" 2>/dev/null` in library/sourced code — `exec` without a command applies EVERY redirection on the line to the CURRENT shell, permanently.
- **Damage:** the `2>/dev/null` silences the CALLING shell's stderr for its lifetime — in the 2026-07-22 forensic incident, one provider launch left the operator's interactive terminal unable to print any error, from any command, until closed. The paired unlock helper had used the correct form all along; the asymmetry was the defect.
- **Mandate (§11.4.67(6)):** in sourced/library functions, brace-scope fd-manipulation `exec : { exec 9>>"$file"; } 2>/dev/null`. A command-less `exec` combining fd operands with additional unbraced redirections in sourced code is a review defect and the greppable class of gate `CM-SHELL-EXEC-REDIRECTION-SCOPED` (brace-scoped forms and `exec` WITH a command are exempt — the §11.4.201(1) false-positive guard).
- **Marker probe (the class's verification pattern):** echo a marker to stderr AFTER calling the function; the marker MUST appear on the terminal. Absent ⇒ the function ate the shell's stderr.

## Sources / evidence

- Control-experiment demonstrations (I1, I2, I3, I4): 2026-07-23, HEL-010 constitution-promotion session — transcript-captured runs ( `timeout` on a function → `rc=127; pgrep -fa unique-needle self-match; column-0- }` heredoc truncation dropping the tail sentinel; one-line xtrace then blindness after `exec 2>/dev/null` ).
- Control-experiment demonstration (I7): run 2026-07-22T20:36Z UTC, transcript-captured 20:41Z (2026-07-23 local), HEL-010 harvest delta — hermetic repro, 3/3 deterministic iterations, four scenarios (naive `$(...)` stall == full budget;

same probe to file ~0; fd-redirected watchdog ~0 with output intact; wedged probe still bounded rc=124), field incident first measured 2026-07-23 by a consuming toolchain's install re-review (cmd-subst read 15s == budget vs file-write 0s).

- Forensic incidents: 2026-07-22 sourced- `exec` `stderr` silencing + its xtrace-blinded investigation (§11.4.67(6) FACT); 2026-07-13 `pgrep -f` `build-guard` false-refusal (§11.4.201 forensic anchor); 2026-07-17 carrier/measurement harvest (I5, I6 — §11.4.201(6)(7) FACTs).
- Export note (§11.4.65, honest): `.html` / `.pdf` twins for this document are NOT generated in this commit — the governance-twin exporter is not yet a documented runnable in-repo script (the Rev-60 probed finding); twins are owed to the exporter commit, no silent divergence claimed (§11.4.106(E)).